Pearl Air Quality Index (AQI) Forecasting System

End-to-End Machine Learning Project (MLOPS)

Submitted By

Ibtisam Asghar

Organization

10 Pearls

Date

6 June 2026

Contents

1. Introduction

Air pollution is a major problem that harms human health. The Air Quality Index (AQI) is a standard tool used to show how clean or polluted the air is. High AQI scores mean the air is unhealthy, especially for children, old people, and anyone with breathing issues. In Islamabad, Pakistan, the air quality changes quickly due to different seasons, traffic, and weather.

The Pearls AQI Predictor is a machine learning system built to forecast AQI levels in Islamabad for the next 3 days (24, 48, and 72 hours). The goal of this work is to build an automated, production-grade system entirely on a $0 budget using free cloud services.

2. Project Summary

2.1 High-Level Description

The Pearls AQI Predictor is a serverless machine learning application. It automates the entire system lifecycle using these free cloud tiers:

1. **Continuous Data Ingestion:** Collects weather and pollution data every hour.
2. **Serverless Feature Store:** Stores raw and processed data in a free MongoDB Atlas database.
3. **Incremental Feature Engineering:** Calculates rolling averages and past data trends every hour.
4. **Daily Retraining:** Retrains machine learning models every night using free GitHub Actions runners.
5. **Champion-Challenger Evaluation:** Compares new models against old ones and only keeps the best-performing model.
6. **Model Registry:** Saves model files on the Hugging Face Hub for free.
7. **FastAPI Prediction Service:** Delivers quick predictions (under 5ms) over the web.
8. **Streamlit Dashboard:** Displays air quality forecasts, charts, and health tips in a clean, dark-themed website.

2.2 System Overview

The system runs by itself in a continuous loop:

1. **Collect:** An hourly script fetches live weather and air quality data.
2. **Process:** The system updates features like rolling averages and saves them to MongoDB.
3. **Train:** Every night, a script pulls the data, trains three predictive models, and checks their accuracy.

4. **Compare:** If the new model is more accurate than the old one, it becomes the new "Champion".
5. **Save:** The system uploads model files to Hugging Face and logs details in MongoDB.
6. **Serve:** A FastAPI service loads the champion model to answer live prediction requests.
7. **Display:** The Streamlit dashboard displays the final forecasts to users.

3. Data Collection

3.1 Data Sources

The system uses two public APIs to collect data:

1. **Open-Meteo API (Primary):** Collects hourly weather data (temperature, humidity, wind) and pollutant levels ($PM_{2.5,}PM_{10,}NO_{2,}SO_{2,}CO_{,}O_{3,}$)
2. **AQICN API (Fallback):** Used as a backup. If Open-Meteo fails or hits limit rules, the system pulls current $PM_{2.5}$ data from AQICN.

3.2 Historical Data (Backfill)

The system uses a script (**src/pipelines/backfill.py)** to gather historical data for training:

1. It downloads past data from January 2025 onwards.
2. It matches weather data with pollution data using hourly timestamps in local Pakistan time.
3. It creates future target columns (target_aqi_24h, target_aqi_48h, target_aqi_72h) by shifting the AQI column backward.
4. It cleans up missing rows and uploads everything to MongoDB.

4. Feature Engineering

4.1 AQI Calculation

The system converts raw $PM_{2.5}$ dust levels ($\mu g/m^3$) into the standard US EPA Air Quality Index (AQI) using this formula:

$$AQI = \frac{I_{high} - I_{low}}{C_{high} - C_{low}} \times (C - C_{low}) + I_{low}$$

The system uses the standard US EPA breakpoint categories:

PM2.5 Range (µg/m3)	AQI Range	Health Category
0.0 to 12.0	0 to 50	Good
12.1 to 35.4	51 to 100	Moderate
35.5 to 55.4	101 to 150	Unhealthy for Sensitive Groups
55.5 to 150.4	151 to 200	Unhealthy
150.5 to 250.4	201 to 300	Very Unhealthy
250.5 to 500.4	301 to 500	Hazardous

4.2 Features for Modeling

The feature script (**src/pipelines/feature_pipeline.py**) creates specific columns to help the machine learning models learn better:

Cyclical Time Features

Hours, days, and months repeat in cycles. To help the model understand that hour 23 is close to hour 0, timestamps are turned into sine and cosine curves:

$$Sin_{hour} = \sin\left(\frac{2\pi \times hour}{24}\right) \ , Sin_{day} = \sin\left(\frac{2\pi \times Weekday}{7}\right) , Sin_{month} = \sin\left(\frac{2\pi \times month}{12}\right)$$

$$cos_{hour} = \cos\left(\frac{2\pi \times hour}{24}\right) \ , cos_{day} = \cos\left(\frac{2\pi \times Weekday}{7}\right) , cos_{month} = \cos\left(\frac{2\pi \times month}{12}\right)$$

Time Lags & Rolling Windows

1. **Lags:** The system adds values from 1, 2, 24, and 48 hours ago to show recent air quality trends.
2. **Rolling Averages:** It computes moving averages and standard deviations over 3, 12, and 24-hour blocks to smooth out random spikes.

AQI Change Rate

This formula measures how fast the pollution index is changing hour by hour. A tiny number ($\in = 10^{-5}$) is added to prevent mathematical errors if the previous AQI was zero:

$$\text{aqi\_change\_rate}_{1h} = \frac{AQI_t - AQI_{t-1}}{AQI_{t-1} + 10^{-5}}$$

5. Model Training

5.1 Training Data

The training script (**src/pipelines/training_pipeline.py**) pulls all historical logs from MongoDB. It sorts everything by time and splits the data chronologically:

1. **Training Set:** The oldest **80%** of the data.
2. **Validation Set:** The newest **20%** of the data.

The system avoids random splitting to prevent data leakage. This ensures the model is always tested on how well it can predict the future using only past data.

5.2 Models and Metrics

The system trains three distinct RandomForestRegressor models. Each model handles one specific future time window:

1. **Model 1:** Predicts AQI in +24 hours.
2. **Model 2:** Predicts AQI in +48 hours.
3. **Model 3:** Predicts AQI in +72 hours.

All models use standard settings (max_depth=15, n_estimators=100, random_state=42) to keep results consistent. The system tracks Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the R-Squared score (R^2).

5.3 Daily Model Training Pipeline

An automated script runs every day to execute the following steps:

1. Download updated data from MongoDB.
2. Clean and scale the data using a training data pipeline.
3. Train the three Random Forest models.
4. Calculate validation metrics using the test data split.
5. Save the trained files (model.pkl, preprocessor.pkl, features.pkl) to disk.

5.4 Model Registry

The system manages models across two free layers:

- **Hugging Face Hub:** Stores the large model weight files (.pkl) in an organized folder structure.
- **MongoDB Atlas:** Acts as a metadata log. It saves hyperparameter values, errors, and Hugging Face file paths.

Champion Logic

The training script compares the average RMSE of the new candidate model against the active champion model.

$$Average\ RMSE = \frac{RMSE_{24h} + RMSE_{48h} + RMSE_{72h}}{3}$$

If the new model has a lower error score, it is tagged as is_champion: true, and the old champion flag is turned off.

6. Prediction Pipeline

The API layer (src/api/main.py) uses FastAPI to deliver predictions.

Startup Behavior

When the API server boots up, it checks MongoDB for the active champion model, downloads those specific files from Hugging Face, and loads them directly into memory for fast performance.

API Endpoints

Endpoint	HTTP Method	Function / Core Task
/health	GET	Checks if the server and database connections are running normally.

Endpoint	HTTP Method	Function / Core Task
/metrics	GET	Returns accuracy scores (MAE, RMSE, R^2) for the active champion model.
/predict/live	GET	Grabs the newest hourly data row from MongoDB and instantly predicts the AQI for the next 24, 48, and 72 hours.
/predict/custom	POST	Accepts manual input data sent by a user in JSON format and returns a custom prediction.

7. Dashboard (Streamlit)

The frontend website is built with Streamlit **(src/dashboard/app.py)** and includes the following features:

- **Main Display:** Shows the current live AQI score in Islamabad with a clean color indicator matching the US EPA scale, along with specific health safety recommendations.
- **Forecast Panel:** Displays a 3-day projection trend using interactive line charts.
- **Explainable AI (SHAP):** Uses SHAP charts to show exactly which features (like wind speed or gas levels) caused the model to predict a higher or lower AQI score.
- **Data History:** Offers sliders to view up to 90 days of historical charts, pollutant correlations, and daily pollution cycles.
- **Fallback Design:** If the FastAPI server goes offline, the dashboard automatically connects directly to MongoDB and Hugging Face to calculate predictions locally, ensuring the site stays functional.

8. Automation with GitHub Actions

8.1 Workflow Summary

Two automated GitHub Actions files run the live system without human intervention:

Workflow File	Run Frequency	Core Task
feature_pipeline.yml	Every hour (at minute 5)	Fetches current API data, calculates features, and saves them to MongoDB.
training_pipeline.yml	Every day at 2:00 AM UTC	Retrains models, runs champion validation checks, and saves files to Hugging Face.

Secure credentials like database connection string links and API keys are stored in GitHub Repository Secrets to keep them hidden from the public.

9. Deployment Details

9.1 Platform Infrastructure Summary

The entire deployment architecture is distributed across three free cloud components:

Component Platform	Service Tier Used	Operational Role	Key Settings / Security
MongoDB Atlas	Free Tier (M0 Sandbox)	Stores live features and logs model metadata.	Compound index on {timestamp: 1, location: 1}; Network access (0.0.0.0/0) for cloud runners.
Streamlit Cloud	Free Public Hosting	Hosts the user-facing web dashboard app.	App credentials saved securely inside Streamlit's private settings panel.
Hugging Face	Free Spaces / Repos	Hosts model weights and hosts the FastAPI container.	Runs a Docker environment on port 7860 for handling public REST requests